Webpack Interview & Development Cheat Sheet

Part 2 — Assets, Development Workflow, Code Splitting & Optimization

> **Primary reference:** Your uploaded Webpack course notes. Modern Webpack 5 practices are noted where they differ.

Asset Handling

Why Assets Need Webpack

Without Webpack:

```
HTML
 ├── image.png
 ├── styles.css
 ├── font.woff2
```

You manage everything manually.

With Webpack:

```
Asset
  |
  ▼
Dependency Graph
  |
  ▼
Optimized Output
```

Memory

> Everything becomes a **module**.

Asset Modules (Webpack 5)

Concept & Example	Quick Reference
`asset/resource`	Emits a separate file and returns its URL. **Use for:** images, fonts, videos.
`asset/inline`	Converts asset into Base64 and embeds in JS. **Best:** tiny icons.
`asset/source`	Imports file contents as a string. **Example:** Markdown, SVG source.
`asset`	Automatically chooses inline vs resource depending on size. Recommended default.

Example

```
{
  test:/\.(png|jpg|svg)$/,
  type:"asset"
}
```

Memory:

> **Resource → File**

> **Inline → Base64**

> **Source → String**

Images

```
import logo from "./logo.png";
```

Webpack returns

```
"/assets/logo.823af.png"
```

instead of binary data.

Benefits

- Hashing
- Optimization
- Cache busting

Fonts

```
import "./fonts.css";
```

Webpack copies fonts automatically.

Typical extensions

```
woff2
woff
ttf
eot
```

CSS Assets

```
background:url("./hero.jpg");
```

Webpack automatically resolves the dependency.

No manual copying.

Source Maps

Why?

Production bundle

```
bundle.js
```

is unreadable.

Need mapping

```
Original Source

↓

Source Map

↓

Bundle
```

Common Source Maps

Option	Use
`eval-source-map`	Fastest development
`inline-source-map`	Good debugging
`source-map`	Production
`hidden-source-map`	Production error reporting

Interview

Why not always use source-map?

Large files + exposes source code.

Development Workflow

Watch Mode

```
webpack --watch
```

Automatically rebuilds.

No browser refresh.

Memory

> Watch = Build

---

webpack-dev-server

```
npm run serve
```

Provides

- Live server
- Automatic rebuild
- Browser refresh
- HMR support

Course note:

Development bundles stay **in memory**, not on disk.

Memory

> Dev Server = Build + Serve

Hot Module Replacement (HMR)

Instead of

```
Reload entire page
```

only changed module updates.

Example

`Button.js changed`

↓

```
Button replaced
```

↓

```
App keeps running
```

Benefits

- Faster development
- Preserve state
- Better productivity

Interview

Difference

Live Reload

↓

Entire page reloads

HMR

↓

Only changed module reloads

Code Splitting

Why?

Instead of

```
5 MB bundle
```

load

```
Login
```

```
↓
Dashboard
↓
Charts
```

only when needed.

Memory

> Load less.

Load later.

Dynamic Imports

```
import("./Dashboard")
```

returns

```
Promise
```

Webpack creates a new chunk automatically.

Lazy Loading

Load feature

Only

When user needs it.

Examples

- Settings page
- Admin panel
- Reports

- Charts

Interview

Difference

Code Splitting

↓

Creates chunks

Lazy Loading

↓

Loads chunks later

Chunk

Small piece of bundle.

Example

`main.js`

`vendor.js`

`dashboard.js`

Runtime Chunk

Contains

- Module loading logic
- Chunk loading logic

Keeps runtime separate from application code.

Benefits

Better caching.

SplitChunksPlugin

Purpose

Extract shared code.

Instead of

```
React copied
```

```
React copied
```

```
React copied
```

Generate

```
vendor.js
```

shared everywhere.

Example

```
optimization:{
 splitChunks:{
   chunks:"all"
 }
}
```

Interview

Why split vendor code?

Browser cache changes less often.

Caching

Browser Caching

User downloads

```
vendor.js
```

once.

Next visit

↓

Reuse cached copy.

Content Hash

```
bundle.a23f81.js
```

Hash changes

Only

When file changes.

Example

```
filename:"[name].[contenthash].js"
```

Memory

> Same content

↓

Same filename

Cache Busting

Without hash

`bundle.js`

Browser may use stale version.

With hash

`bundle.9ad821.js`

Always fresh.

Optimization

Tree Shaking

Removes

Unused exports.

Requires

- ES Modules
- Production mode

Example

```
export function a(){}
export function b(){}
```

Only

```
a()
```

used.

Webpack removes

`b()`

Memory

> Dead code removal.

Side Effects

```
{
 "sideEffects":false
}
```

Tells Webpack

Unused imports

can be removed safely.

Interview

Why important?

Enables aggressive tree shaking.

Minification

Production mode

Automatically

- Removes whitespace
- Shortens variables
- Compresses code

No plugin needed.

Performance Optimization

Technique	Benefit
Code Splitting	Smaller initial download
Tree Shaking	Removes dead code
Content Hash	Better browser caching
SplitChunks	Shared dependencies
Lazy Loading	Faster first paint

Technique	Benefit
Asset Modules	Smaller asset pipeline
Minification	Smaller bundles
HMR	Faster development

Bundle Analyzer

Plugin

Visualizes bundle.

Example output

```
bundle
├── React
├── Lodash
├── Charts
└── App
```

Useful for

- Large libraries
- Duplicate packages
- Bundle size reduction

Development vs Production

Development	Production
Fast builds	Optimized builds
Source Maps	Minified

Development	Production
HMR	Tree Shaking
Easier debugging	Smaller bundles
Better DX	Better UX

Memory

Development

↓

Developer

Production

↓

Customer

Common Mistakes

Mistake	Fix
Missing loader	Add appropriate loader
Wrong loader order	Remember **right → left**
Huge bundle	SplitChunks + Dynamic Imports
No caching	Use `[contenthash]`
Images not loading	Check Asset Modules/publicPath
Large vendor bundle	Analyze bundle
Importing entire library	Import only required modules
Source maps in production	Use hidden or disable

Frequently Asked Interview Questions

Question	Answer

Question	Answer
Code Splitting vs Lazy Loading?	Splitting creates chunks. Lazy loading fetches them later.
Why Dynamic Imports?	Automatic async chunk generation.
What is HMR?	Replace changed modules without full refresh.
Why SplitChunks?	Shared dependency extraction.
Why Content Hash?	Cache busting.
Tree Shaking requirements?	ES Modules + Production mode.
Why Source Maps?	Debug minified code.
Why Bundle Analyzer?	Identify large dependencies.

Best Practices

✅ Prefer dynamic imports for large pages.

✅ Never ship development builds.

✅ Keep initial bundle small.

✅ Use `asset` modules instead of deprecated loaders.

✅ Always use content hashes.

✅ Enable tree shaking.

✅ Analyze bundle size regularly.

✅ Cache vendor libraries.

✅ Lazy load infrequently used routes.

Covered from Your Reference

✓ Image handling

✓ CSS handling

✓ webpack-dev-server

✓ Watch Mode

✓ Code Splitting

✓ Dynamic Imports

✓ Vendor Caching

✓ Chunk Hashing

✓ Cache Busting

✓ Production Builds

✓ Development Builds

✓ Performance concepts

✓ Asset handling

✓ Deployment preparation

Based on your uploaded notes.

Modern Topics Added

These are important for Webpack 5 but are not fully covered in the course:

- ✅ Asset Modules
- ✅ Runtime Chunk
- ✅ Side Effects
- ✅ Bundle Analyzer
- ✅ Modern Tree Shaking behavior
- ✅ Updated optimization practices

In **Part 3**, we'll cover **Module Federation, Environment Variables, Multiple Configurations,** `webpack-merge` **, Externals, Deployment, Debugging, Security, Production Best Practices, Common Errors, and a one-page Interview Revision Sheet**. That final part will complete the cheat sheet while keeping it compact enough for quick interview review.